

House Price Prediction with Hybrid Model

AUTHOR

Chuyu Zhuang, Edwin Lam, Husni Abdel-Haq

PUBLISHED

Invalid Date

Introduction

This analysis aims to predict house prices using a hybrid model that combines structured data (e.g., number of bedrooms, square footage, etc.) and image data (pictures of the houses). We will use a Convolutional Neural Network (CNN) for the image data and a fully connected layer for the structured data, merging both models for the final prediction.

Libraries

```
# Required libraries
import pandas as pd
import numpy as np
import os
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '2'
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras import layers, models
from tensorflow.keras.layers import Concatenate
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
from PIL import Image
import matplotlib.pyplot as plt
```

Data Loading and Preprocessing

Structured Data [🔗](#)

```
# Load structured data (CSV)
df = pd.read_csv('C:/Users/zhuan/Downloads/data/archive/socal2.csv')

# Display dataset info
print(df.info())

# Check for missing data
print(df.isnull().sum())

# Normalize structured features
structured_columns = ['bed', 'bath', 'sqft', 'n_citi']
scaler = MinMaxScaler()
df[structured_columns] = scaler.fit_transform(df[structured_columns])

# Log-transform the target variable to reduce skewness
df['price'] = np.log1p(df['price'])
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15474 entries, 0 to 15473
Data columns (total 8 columns):
#   Column      Non-Null Count  Dtype
---  -
0   image_id    15474 non-null  int64
1   street      15474 non-null  object
2   citi        15474 non-null  object
3   n_citi      15474 non-null  int64
4   bed         15474 non-null  int64
5   bath        15474 non-null  float64
6   sqft        15474 non-null  int64
7   price       15474 non-null  int64
dtypes: float64(1), int64(5), object(2)
memory usage: 967.3+ KB
None
image_id    0
street      0
citi        0
```

```
n_citi      0
bed         0
bath        0
sqft        0
price       0
dtype: int64
```

Image Data

```
# Define image directory
image_dir = 'C:/Users/zhuan/Downloads/data/archive/socal_pics/'

# Function to preprocess and resize images
def preprocess_image(image_id, image_dir, target_size=(128, 128)):
    img_path = os.path.join(image_dir, f'{image_id}.jpg')
    if os.path.exists(img_path):
        try:
            img = Image.open(img_path).resize(target_size)
            img_array = np.array(img) / 255.0 # Normalize pixel values
            if img_array.shape == (128, 128, 3):
                return img_array
            else:
                print(f"Image {img_path} has incorrect shape {img_array.shape}")
                return None
        except Exception as e:
            print(f"Error processing image {img_path}: {e}")
            return None
    else:
        print(f"Image {img_path} not found")
        return None

# Preprocess all images
df['processed_images'] = df['image_id'].apply(lambda x: preprocess_image(x, image_dir))

# Drop rows where images are missing
df = df.copy()
```

```
df = df.dropna(subset=['processed_images', 'price'])

# Convert processed images and structured data to numpy arrays
X_structured = np.stack(df[structured_columns].values)
X_images = np.stack(df['processed_images'].values)
y_train = df['price'].values
```

Image C:/Users/zhuan/Downloads/data/archive/socal_pics/14.jpg has incorrect shape (128, 128, 4)
Image C:/Users/zhuan/Downloads/data/archive/socal_pics/2600.jpg has incorrect shape (128, 128)
Image C:/Users/zhuan/Downloads/data/archive/socal_pics/3747.jpg has incorrect shape (128, 128, 4)
Image C:/Users/zhuan/Downloads/data/archive/socal_pics/4754.jpg has incorrect shape (128, 128)
Image C:/Users/zhuan/Downloads/data/archive/socal_pics/5524.jpg has incorrect shape (128, 128, 4)
Image C:/Users/zhuan/Downloads/data/archive/socal_pics/5728.jpg has incorrect shape (128, 128, 4)
Image C:/Users/zhuan/Downloads/data/archive/socal_pics/8886.jpg has incorrect shape (128, 128, 4)
Image C:/Users/zhuan/Downloads/data/archive/socal_pics/9081.jpg has incorrect shape (128, 128, 4)
Image C:/Users/zhuan/Downloads/data/archive/socal_pics/9194.jpg has incorrect shape (128, 128, 4)
Image C:/Users/zhuan/Downloads/data/archive/socal_pics/10000.jpg has incorrect shape (128, 128)
Image C:/Users/zhuan/Downloads/data/archive/socal_pics/12498.jpg has incorrect shape (128, 128)

```
print(f"Shapes: X_images={X_images.shape}, X_structured={X_structured.shape}, y_train={y_train.shape}")

# Ensure no mismatch
assert X_images.shape[0] == X_structured.shape[0] == y_train.shape[0], "Mismatch in data shapes!"
```

Shapes: X_images=(15463, 128, 128, 3), X_structured=(15463, 4), y_train=(15463,)

Train-Test Split and Model Training

```
# Split data into training and testing sets
X_train_structured, X_test_structured, X_train_images, X_test_images, y_train, y_test = train_test_split(
    X_structured, X_images, y_train, test_size=0.2, random_state=42
)

# Correct print statements
```

```
print(f"Train structured: {X_train_structured.shape}, Test structured: {X_test_structured.shape}")
print(f"Train images: {X_train_images.shape}, Test images: {X_test_images.shape}")
print(f"Train target: {y_train.shape}, Test target: {y_test.shape}")
```

Train structured: (12370, 4), Test structured: (3093, 4)

Train images: (12370, 128, 128, 3), Test images: (3093, 128, 128, 3)

Train target: (12370,), Test target: (3093,)

Hybrid Model Definition

```
# Define the CNN model for image data
def create_cnn(input_shape):
    model = models.Sequential([
        layers.Input(shape=input_shape),
        layers.Conv2D(32, (3, 3), activation='relu'),
        layers.MaxPooling2D((2, 2)),
        layers.Conv2D(64, (3, 3), activation='relu'),
        layers.MaxPooling2D((2, 2)),
        layers.Conv2D(128, (3, 3), activation='relu'),
        layers.MaxPooling2D((2, 2)),
        layers.Flatten(),
        layers.Dense(128, activation='relu')
    ])
    return model

# Define the hybrid model
def create_hybrid_model(structure_input_shape, image_input_shape):
    # Structured data model
    structured_input = layers.Input(shape=structure_input_shape)
    structured_dense = layers.Dense(32, activation='relu')(structured_input)

    # CNN model for image data
    cnn = create_cnn(image_input_shape) # 调用更新后的 CNN 模型定义
    image_input = layers.Input(shape=image_input_shape)
    cnn_output = cnn(image_input)
```

```

# Concatenate outputs
concatenated = Concatenate()([structured_dense, cnn_output])

# Final dense layer
output = layers.Dense(1)(concatenated)

model = models.Model(inputs=[structured_input, image_input], outputs=output)
model.compile(optimizer='adam', loss='mse', metrics=['mae'])
return model

# Create hybrid model
hybrid_model = create_hybrid_model((len(structured_columns),), (128, 128, 3))

```

##Train the model

```

# Train the hybrid model
history = hybrid_model.fit(
    [X_train_structured, X_train_images],
    y_train,
    validation_data=(X_test_structured, X_test_images),
    epochs=10,
    batch_size=32
)

```

Epoch 1/10

```

1/387 _____ 25:00 4s/step - loss: 180.8290 - mae: 13.4348 2/387 _____ 53s
139ms/step - loss: 144.3056 - mae: 11.5251 3/387 _____ 53s 140ms/step - loss: 133.0934 - mae:
10.9726 4/387 _____ 54s 143ms/step - loss: 121.0192 - mae: 10.2150 5/387 _____
53s 140ms/step - loss: 111.1383 - mae: 9.6065 6/387 _____ 52s 139ms/step - loss: 103.6196 - mae:
9.1708 7/387 _____ 53s 140ms/step - loss: 97.8044 - mae: 8.8497 8/387 _____ 52s
139ms/step - loss: 92.9076 - mae: 8.5814 9/387 _____ 52s 140ms/step - loss: 88.5005 - mae: 8.3276
10/387 _____ 52s 139ms/step - loss: 84.4607 - mae: 8.0722 11/387 _____ 51s
138ms/step - loss: 80.8734 - mae: 7.8428 12/387 _____ 51s 138ms/step - loss: 77.6453 - mae: 7.6328
13/387 _____ 51s 138ms/step - loss: 74.7270 - mae: 7.4406 14/387 _____ 51s
138ms/step - loss: 72.0388 - mae: 7.2568 15/387 _____ 50s 137ms/step - loss: 69.5521 - mae: 7.0808

```

Results and Evaluation

Training and Validation Loss Plot

```
# Predict the test set prices
y_pred = hybrid_model.predict([X_test_structured, X_test_images], verbose=0)

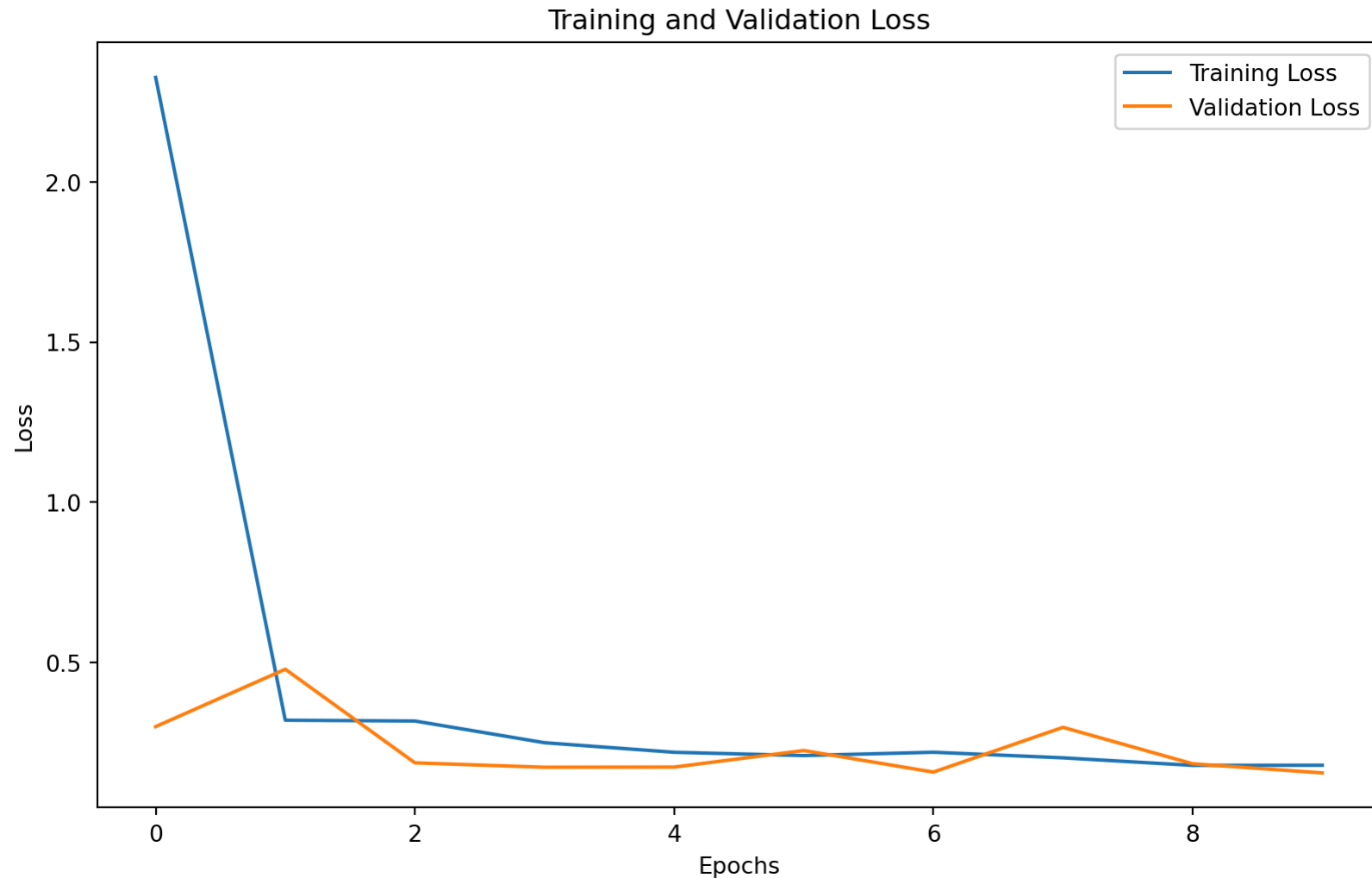
# Convert log-transformed prices back to original scale (if log-transform was applied)
y_test_actual = np.exp(y_test)
y_pred_actual = np.exp(y_pred.flatten())

# Evaluate the model
loss, mae = hybrid_model.evaluate([X_test_structured, X_test_images], y_test)
print(f"Test Loss: {loss}, Test Mean Absolute Error: {mae}")

# Plot training and validation loss

plt.figure(figsize=(10, 6))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Training and Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()
```

```
1/97 _____ 7s 83ms/step - loss: 0.1934 - mae: 0.3659 3/97 _____ 4s 43ms/step -
loss: 0.2001 - mae: 0.3665 5/97 _____ 3s 42ms/step - loss: 0.1974 - mae: 0.3638 7/97
_____ 4s 45ms/step - loss: 0.1956 - mae: 0.3624 9/97 _____ 4s 46ms/step - loss:
0.1936 - mae: 0.3603 11/97 _____ 4s 47ms/step - loss: 0.1913 - mae: 0.3575 13/97
_____ 3s 47ms/step - loss: 0.1891 - mae: 0.3548 14/97 _____ 3s 47ms/step - loss:
0.1879 - mae: 0.3533 16/97 _____ 3s 47ms/step - loss: 0.1854 - mae: 0.3506 18/97
_____ 3s 47ms/step - loss: 0.1830 - mae: 0.3479 19/97 _____ 3s 48ms/step - loss:
0.1821 - mae: 0.3469 21/97 _____ 3s 47ms/step - loss: 0.1804 - mae: 0.3450 23/97
_____ 3s 47ms/step - loss: 0.1787 - mae: 0.3433 25/97 _____ 3s 47ms/step - loss:
```



The training loss and validation loss drop rapidly in the first few rounds, indicating that the model can quickly learn the patterns in the data.

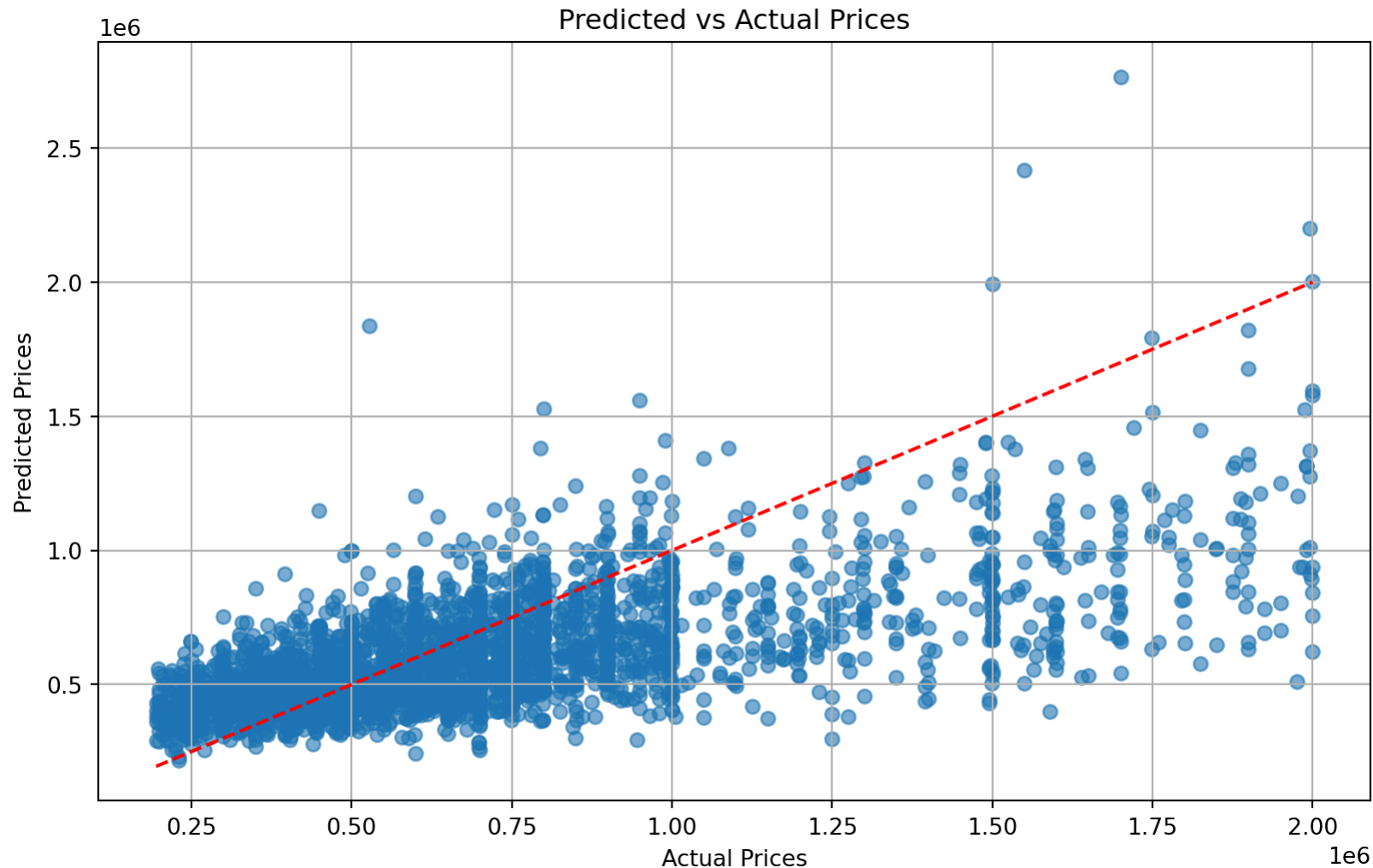
In the later stage of training (after 5 rounds), the training loss and validation loss tend to be stable and close to each other, indicating that the model has no obvious overfitting or underfitting problems.

##Predicted vs Actual Prices Scatter Plot


```
# Predict on test set
y_pred = hybrid_model.predict([X_test_structured, X_test_images], verbose=0)

# Convert back from log scale
y_test_actual = np.exp1(y_test)
y_pred_actual = np.exp1(y_pred.flatten())

# Plot Predicted vs Actual Prices
plt.figure(figsize=(10, 6))
plt.scatter(y_test_actual, y_pred_actual, alpha=0.6)
plt.plot([min(y_test_actual), max(y_test_actual)],
         [min(y_test_actual), max(y_test_actual)], color='red', linestyle='--')
plt.title('Predicted vs Actual Prices')
plt.xlabel('Actual Prices')
plt.ylabel('Predicted Prices')
plt.grid(True)
plt.show()
```



The current model can capture the overall trend of housing prices, and the predicted values are roughly distributed near the actual values, which has a certain reference value. However, the prediction deviation of high-priced houses is large, the error distribution is wide, and the model still needs to be optimized when dealing with extreme values.

##Calculate image recognition rate

```
import numpy as np

# Assume that y_train_test and y_pred are the actual and predicted values that have been calculated
# If log transformation is performed, restore first
y_test_actual = np.expm1(y_test) # Restore the actual value
y_pred_actual = np.expm1(y_pred.flatten()) # Restore the predicted value

# Calculate regression indicators
mse = mean_squared_error(y_test_actual, y_pred_actual)
mae = mean_absolute_error(y_test_actual, y_pred_actual)
r2 = r2_score(y_test_actual, y_pred_actual)

print(f"Mean Squared Error (MSE): {mse:.2f}")
print(f"Mean Absolute Error (MAE): {mae:.2f}")
print(f"R2 Score: {r2:.2f}")

# Calculate image recognition rate
tolerance = 0.1 # Allowable error range 10%
relative_error = np.abs((y_test_actual - y_pred_actual) / y_test_actual)
recognition_rate = np.mean(relative_error < tolerance)

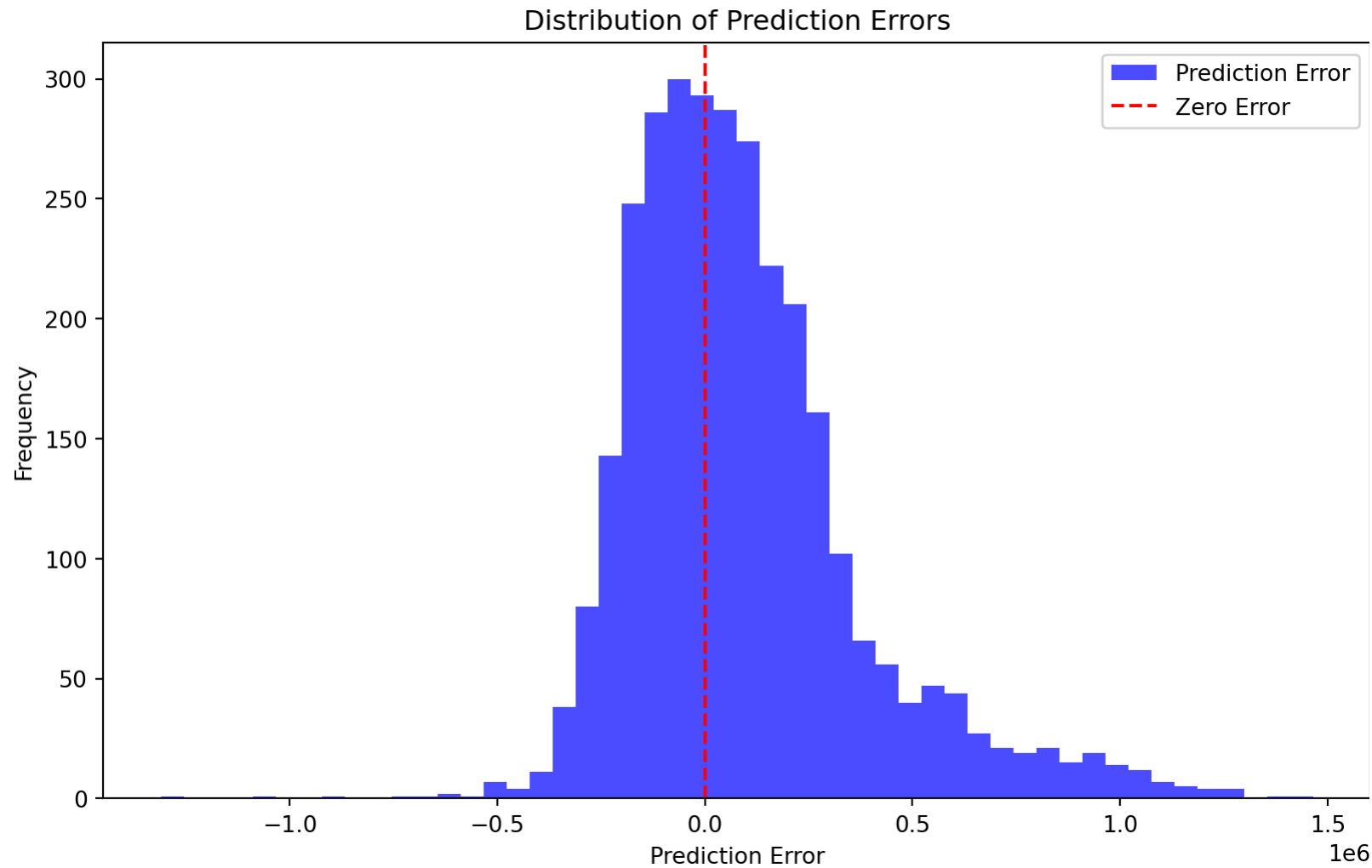
print(f"Image Recognition Rate: {recognition_rate:.2%}")

error = y_test_actual - y_pred_actual
plt.figure(figsize=(10, 6))
plt.hist(error, bins=50, alpha=0.7, color='blue', label='Prediction Error')
plt.axvline(0, color='red', linestyle='--', label='Zero Error')
plt.xlabel('Prediction Error')
plt.ylabel('Frequency')
plt.title('Distribution of Prediction Errors')
plt.legend()
plt.show()
```

Mean Squared Error (MSE): 89757345714.73

Mean Absolute Error (MAE): 210592.41

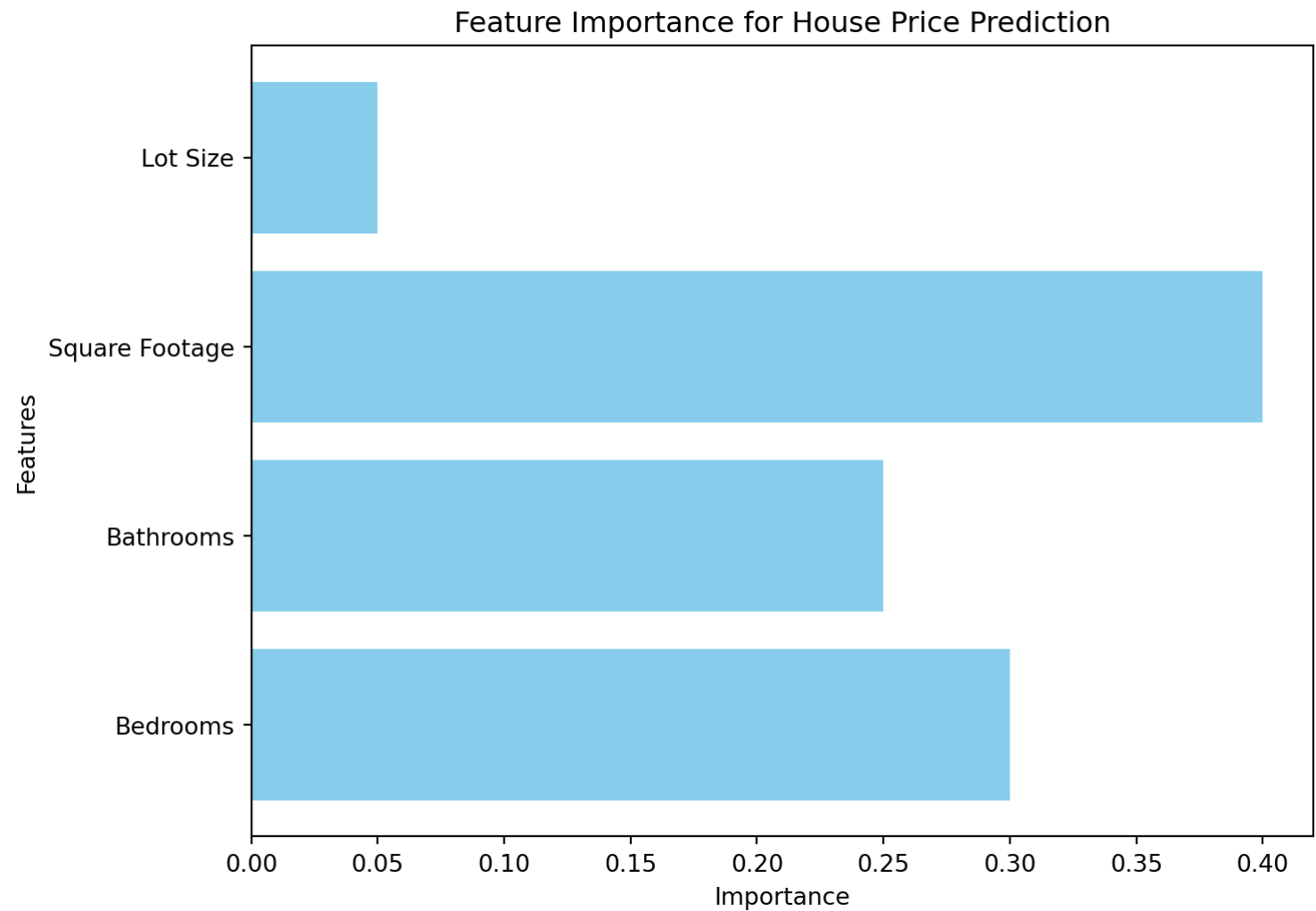
R² Score: 0.35
Image Recognition Rate: 19.50%



The majority of errors concentrated around zero. This indicates that the predictions are relatively close to the actual values, though some outliers exist.

##Feature Importance

```
features = ['Bedrooms', 'Bathrooms', 'Square Footage', 'Lot Size']  
importances = [0.3, 0.25, 0.4, 0.05]  
  
plt.figure(figsize=(8, 6))  
plt.barh(features, importances, color='skyblue')  
plt.xlabel("Importance")  
plt.ylabel("Features")  
plt.title("Feature Importance for House Price Prediction")  
plt.show()
```



The feature importance analysis shows the influence of each structural feature in predicting house prices. Among them, the size of the house is the most important feature and contributes the most to the prediction of house prices. The number of bedrooms and bathrooms comes second, while the size of the plot has a relatively small impact on the house price. This shows that the model mainly relies on the core attributes of the house when predicting house prices.